

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 8

Abonnement

Ce qui se paie, ce qui reste gratuit, et quand

Plans modifiables sans code, paiement simulé, interrupteur d'activation



Périmètre Alanya (Flutter) ■ Alanya-Backend ■ Alanya Admin

Statut Conception — aucune ligne de code écrite

Dépend de Socle de compte (migration 038) ■ Certification (volet 5)

Révisé Volet 5 : la coche n'est plus gratuite, l'abonnement n'est plus dormant

Date 10 septembre 2026

Avant-propos

Le volet 5 avait conçu l'abonnement et le paiement pour les laisser **dormants** : la certification y était gratuite, et les tables commerciales attendaient qu'une offre soit arrêtée. Elle l'est désormais. Ce document la décrit : ce que l'abonnement débloque, ce qu'il coûte, comment on le paie tant qu'aucun agrégateur n'est branché, et comment l'administration passe de « tout gratuit » à « payant » sans prendre les utilisateurs par surprise.

Les décisions qui structurent tout.

- Une offre, deux durées : **250 FCFA par mois** ou **2 000 FCFA par an**.
- Elle débloque la traduction, la sauvegarde, les trajets de confiance, les sonneries par liste — et plus tard la personnalisation du style.
- La **coche indigo** demande deux choses : un abonnement actif *et* une identité vérifiée par un administrateur. Elle tombe à l'échéance.
- Le paiement est **simulé** par un faux agrégateur côté serveur ; le vrai prendra sa place sans rien changer d'autre.
- Un **interrupteur** dans l'administration laisse tout gratuit, puis active le payant avec une période de grâce.
- Les plans — prix, durées, relances, contenu — se modifient **depuis l'administration**, sans passer par le code.

Ce que ce document révisé. Le volet 5 posait « aucune fonctionnalité conditionnée au badge » et « certification gratuite ». La première règle tient toujours — les fonctionnalités s'ouvrent par l'abonnement, jamais par la coche. La seconde est remplacée : hors phase gratuite, la coche exige un abonnement. Le dossier, le coffre à documents et la purge à 90 jours du volet 5 restent valables tels quels.

Méthode. Les affirmations sur l'existant ont été vérifiées en lisant le code des trois dépôts et en interrogeant la base de production. Les chemins cités sont réels.

Table des matières

Avant-propos	1
I Approche naïve	5
1 L’offre	6
1.1 Une offre, deux durées	6
1.2 Ce qui se paie	6
1.3 Ce qui reste gratuit, toujours	6
1.4 Des plans qui se règlent sans développeur	7
2 La coche indigo	8
2.1 Deux conditions, pas une	8
2.2 Ce qu’elle atteste	8
2.3 Quand elle tombe	8
3 Le parcours de l’utilisateur	9
3.1 Le profil met l’offre en avant	9
3.2 S’abonner	10
3.3 Rencontrer une fonctionnalité verrouillée	10
3.4 L’échéance	11
3.5 Le renouvellement automatique	11
3.6 Obtenir la coche	12
4 Du gratuit au payant	13
4.1 Trois phases	13
4.2 Ce que voit l’utilisateur au passage	13
4.3 Les réglages de l’interrupteur	14
5 Le parcours de l’administrateur	15
5.1 L’interrupteur et ses réglages	15
5.2 Les plans	16
5.3 Un utilisateur	16
5.4 Les dossiers d’identité	17
6 Le paiement : simulé aujourd’hui, réel demain	18
6.1 Un faux agrégateur, un vrai parcours	18

6.2 Les stores	18
II Approche technique	19
7 Ce qui existe	20
7.1 Les fonctionnalités à verrouiller	20
8 Modèle de données	21
8.1 Les principes	21
8.2 Migration 080_abonnement.sql	21
8.3 Migration 081_verification.sql	25
9 Droits d'accès	26
9.1 Une seule fonction décide	26
9.2 Ce que reçoit le téléphone	26
9.3 Deux verrous, selon l'endroit où vit la fonctionnalité	27
10 La coche	28
10.1 Une seule écriture	28
10.2 Ce que la coche doit encore corriger dans l'existant	29
11 Le paiement	30
11.1 La couture	30
11.2 Le cycle d'un paiement	30
11.3 Le simulateur	31
11.4 Réconciliation	32
11.5 Les stores et le porte-monnaie	32
12 Échéances et interrupteur	33
12.1 Des jobs posés à la bonne heure	33
12.2 Les notifications	33
12.3 Les transitions de l'interrupteur	34
13 Verrouillage, fonctionnalité par fonctionnalité	35
14 API	37
14.1 Application	37
14.2 Fournisseurs	37
14.3 Administration	38
15 Impact fichier par fichier	39
15.1 Backend	39
15.2 Administration	40

15.3 Application	40
16 Ordre de livraison	42
17 Points de vigilance et questions ouvertes	43
17.1 Décisions prises	43
17.2 Questions ouvertes	43
17.3 Ce qui ne doit pas être oublié	44

PARTIE I

Approche naïve

CHAPITRE 1

L'offre

1.1 Une offre, deux durées

Il n'y a qu'une offre, **Alanya Plus**, vendue sous deux durées :

Durée	Prix	Relance avant l'échéance
Mensuelle	250 FCFA	7 jours avant
Annuelle	2 000 FCFA	30 jours avant — soit 167 F par mois, quatre mois offerts

Les deux durées donnent exactement la même chose. L'annuelle est mise en avant dans l'écran d'offre : elle est moins chère au mois et elle réduit le nombre de renouvellements — donc d'occasions de perdre un abonné pour un simple oubli.

1.2 Ce qui se paie

Fonctionnalité	Ce qu'elle apporte
Traduction des messages	Traduire un message, ou toute une conversation automatiquement, sur le téléphone.
Sauvegarde des données	Sauvegardes chiffrées régulières, sur le téléphone ou sur Google Drive.
Trajets de confiance	Partager sa route à sa liste <i>Confiance</i> et être attendu à l'arrivée.
Sonneries par liste	Une sonnerie propre à la famille, une autre au bureau.
Personnalisation du style	<i>Plus tard.</i> Couleurs et fonds au-delà du clair et du sombre.
La coche 	Le <i>droit</i> à la coche — qui demande en plus une vérification d'identité (chapitre 2).

1.3 Ce qui reste gratuit, toujours

Écrire, appeler, publier un statut, créer un groupe ou une réunion, envoyer des médias : tout ce qui fait d'Alanya une messagerie reste gratuit. On ne fait pas payer la conversation.

Trois cas méritent d'être dits explicitement, parce qu'on pourrait les croire payants :

Reste gratuit	Pourquoi
Suivre le trajet d'un proche	Seul celui qui <i>partage</i> sa route est abonné. Ses cinq proches ne doivent rien payer pour veiller sur lui.
Restaurer une sauvegarde	Faire une <i>nouvelle</i> sauvegarde est payant ; récupérer ses propres données ne l'est jamais.
Le thème clair ou sombre	Il existe déjà, et il relève de l'accessibilité autant que du goût.

Le SOS fait partie de l'offre. Le bouton SOS est réservé aux abonnés, comme les trajets de confiance dont il fait partie. Une exception : un trajet lancé pendant l'abonnement va à son terme avec son SOS, même si l'échéance tombe en route. On ne coupe pas une protection en cours de chemin.

1.4 Des plans qui se règlent sans développeur

Prix, durée, délai de relance, fonctionnalités incluses, mise en avant, disponibilité : tout se modifie depuis l'administration. Deux règles encadrent ces modifications :

- **Un changement de prix ne touche jamais une période déjà payée.** Il s'applique au prochain renouvellement, et l'abonné en est informé dans sa relance.
- **Un plan ne se supprime pas, il se retire.** Il disparaît de l'offre, mais l'historique de ceux qui l'ont payé reste lisible.

Ce que l'administration ne peut pas faire seule. Elle peut rendre une fonctionnalité existante payante ou gratuite, et choisir ce que chaque plan inclut. Elle ne peut pas *inventer* une fonctionnalité : le verrou doit exister dans le code. C'est pourquoi la personnalisation du style figure déjà au catalogue, marquée « à venir » — elle s'activera le jour où l'application saura l'appliquer.

CHAPITRE 2

La coche indigo

2.1 Deux conditions, pas une

La coche  s'affiche quand **les deux** conditions sont réunies :

Identité	Abonnement	Ce qui s'affiche
non vérifiée	actif ou non	aucune coche — l'abonné a ses fonctionnalités
en cours d'examen	actif	aucune coche — « en cours » dans son profil
vérifiée	actif	 la coche
vérifiée	échu	aucune coche — elle revient dès le réabonnement
révoquée	actif ou non	aucune coche, jusqu'à nouvel examen

L'abonnement seul ne suffit pas : sans contrôle, n'importe qui pourrait acheter pour 250 F une coche au nom d'un autre. L'identité seule ne suffit plus : hors phase gratuite, la coche fait partie de l'offre.

2.2 Ce qu'elle atteste

« Alanya a vérifié que la personne derrière ce compte est bien celle dont il porte le nom. » Rien de plus. Ce n'est ni une recommandation, ni une marque de notoriété.

Le sens de la coche change par rapport au volet 5. Le volet 5 réservait la coche personnelle aux *personnalités publiques*, sur preuve de notoriété. Elle devient accessible à *tout abonné* dont l'identité est vérifiée. La pièce de notoriété disparaît donc des pièces demandées ; restent la pièce d'identité et un selfie.

2.3 Quand elle tombe

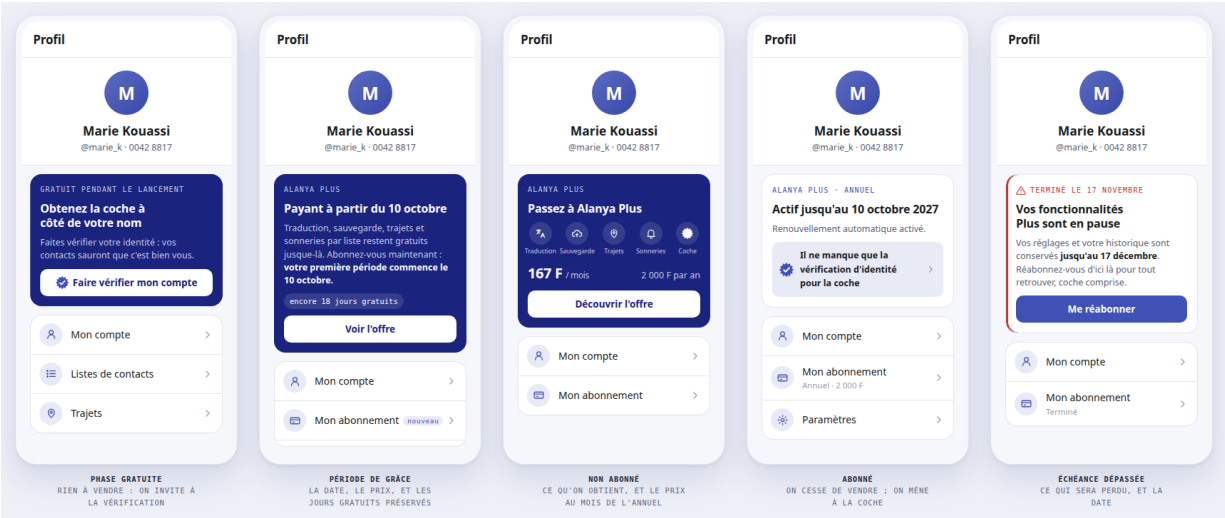
- **À l'échéance**, si l'abonnement n'est pas renouvelé. L'identité, elle, reste vérifiée : se réabonner fait revenir la coche immédiatement, sans nouveau dossier.
- **Sur révocation** par un administrateur, avec un motif — usurpation, fraude. C'est la seule chute qui n'a rien à voir avec l'argent.
- **Quand le compte change de nom.** La vérification portait sur un nom ; le changer retire la coche jusqu'à ce qu'un administrateur ait revu le dossier. Sans cette règle, on achète une coche sous son vrai nom, puis on se renomme « Orange Cameroun ».

CHAPITRE 3

Le parcours de l'utilisateur

3.1 Le profil met l'offre en avant

Une carte se place **juste sous l'en-tête du profil**, avant les contacts préférés. C'est l'emplacement le plus vu de l'écran ; son contenu change avec la situation de l'utilisateur.

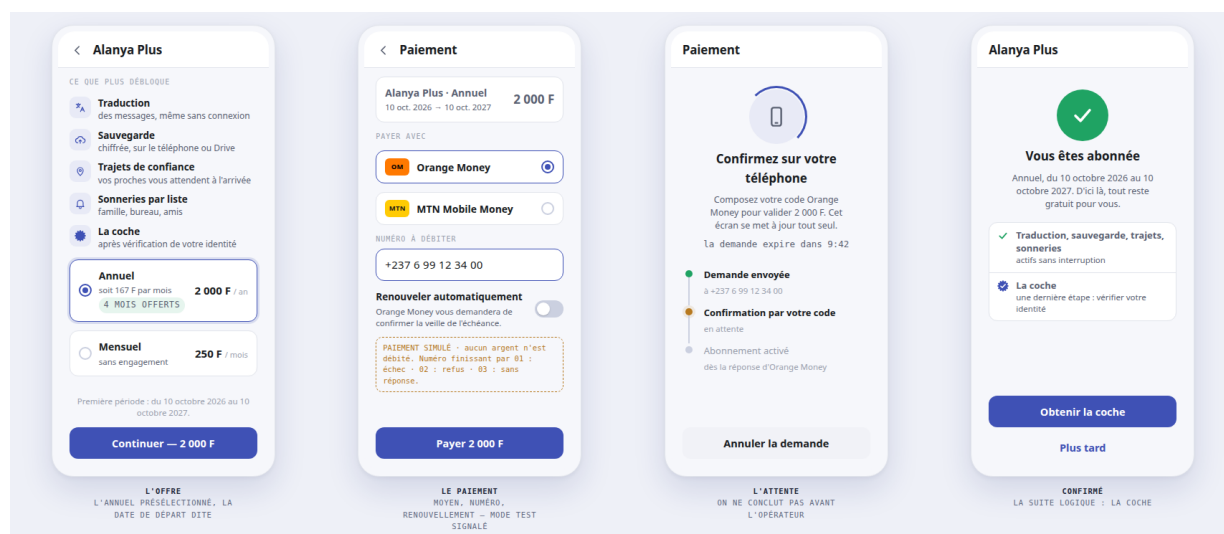


Situation	Ce que dit la carte
Phase gratuite	« Obtenez la coche — gratuite pendant le lancement. » Il n'y a rien à vendre ; on invite à la vérification.
Période de grâce	« Les fonctionnalités Plus deviennent payantes le 12 octobre. » Avec le prix, et un bouton pour s'abonner dès maintenant sans perdre les jours gratuits restants.
Non abonné	« Passer à Alanya Plus » : les cinq icônes des fonctionnalités, « dès 167 F par mois », un bouton.
Abonné	L'échéance, le renouvellement automatique, et — si l'identité n'est pas encore vérifiée — l'invitation à obtenir la coche.
Échéance proche ou dépassée	Le compte à rebours, et ce qui sera supprimé si rien n'est fait.

Une entrée « Mon abonnement » rejoint aussi le menu du profil : la carte attire, l'entrée permet de retrouver ses paiements et de régler le renouvellement.

Maquette interactive, mobile. claude.ai/code/artifact/6ac3fb3e
Source versionnée : docs/architecture/maquettes/abonnement.html ; captures produites par maquettes/_capture_abonnement.py.

3.2 S'abonner



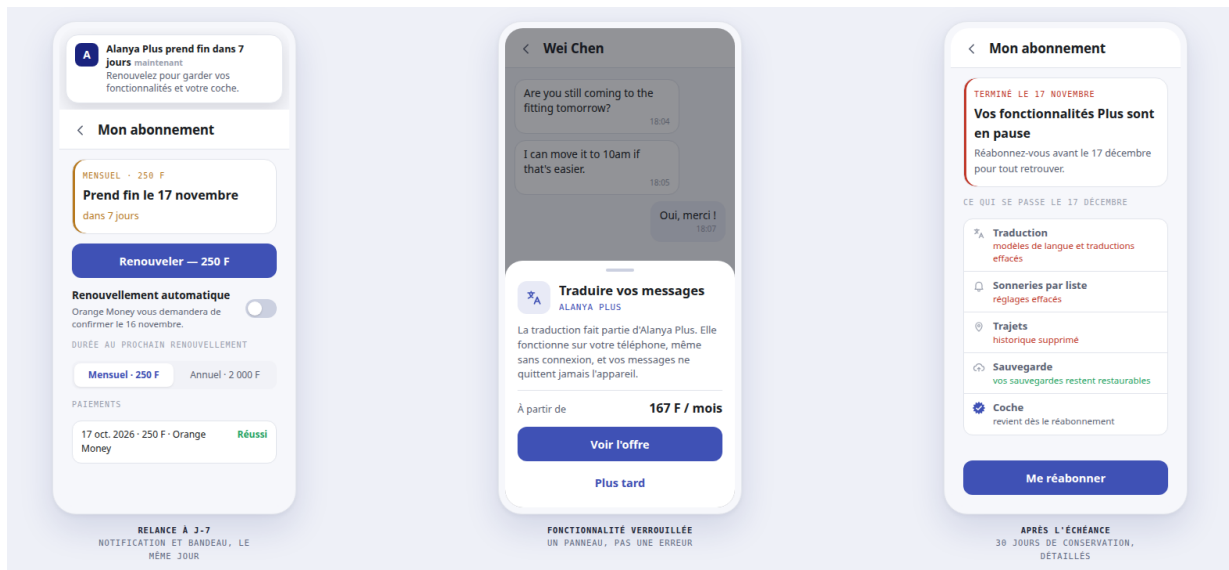
Trois écrans : **l'offre** (les deux durées, l'annuelle présélectionnée, ce qui est inclus), le **paiement** (Orange Money ou MTN MoMo, le numéro à débiter), puis **l'attente de confirmation** — l'utilisateur valide sur son téléphone avec son code secret, l'écran se met à jour de lui-même.

Pourquoi l'écran attend au lieu de conclure. Un paiement mobile money n'est jamais confirmé au moment où on le demande : il l'est quand l'opérateur prévient Alanya, quelques secondes ou minutes plus tard. L'écran le montre tel quel plutôt que d'afficher un succès qui pourrait se révéler faux. Le paiement simulé imite ce délai, pour que l'écran soit éprouvé avant l'arrivée du vrai agrégateur.

3.3 Rencontrer une fonctionnalité verrouillée

Toucher une fonctionnalité payante sans abonnement n'affiche pas d'erreur : un panneau monte du bas de l'écran, dit ce que la fonctionnalité apporte, et propose l'offre. Refermer le panneau ramène exactement où l'on était.

3.4 L'échéance



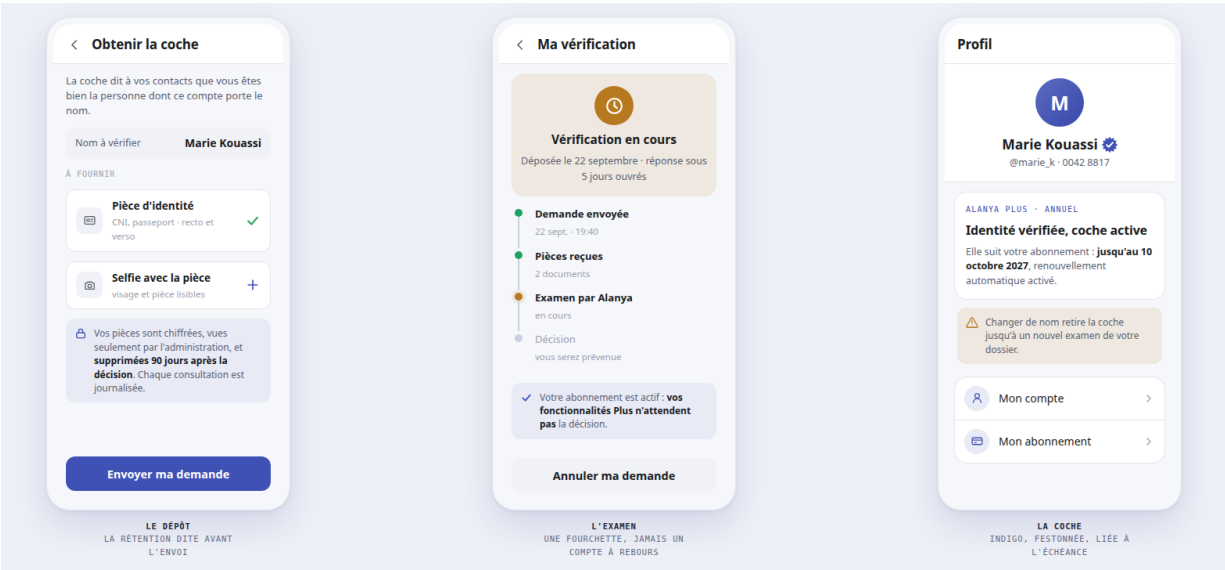
- **La relance** arrive 7 jours avant la fin d'un abonnement mensuel, 30 jours avant celle d'un annuel — par notification et par la carte du profil.
- **À l'échéance**, les fonctionnalités payantes s'arrêtent et la coche tombe. Rien n'est encore effacé.
- **Pendant N jours** (réglables, 30 par défaut), les données restent en place : se réabonner rend tout, exactement comme avant.
- **Une semaine avant la suppression**, un dernier avertissement.
- **Au bout de N jours**, les données propres aux fonctionnalités payantes sont supprimées — le détail, fonctionnalité par fonctionnalité, est en partie II.

3.5 Le renouvellement automatique

L'utilisateur peut l'activer à tout moment dans « Mon abonnement ». Ce qu'il fait réellement dépend du moyen de paiement :

Moyen	Ce qui se passe à l'échéance
Païement simulé	Le renouvellement réussit tout seul — c'est ce qui permet de tester le parcours complet dès maintenant.
Mobile money	L'opérateur envoie une demande de paiement la veille ; l'utilisateur la valide avec son code. Un vrai prélèvement sans action n'existe pas sur mobile money.
Porte-monnaie Alanya	<i>Plus tard.</i> Débit direct du solde, sans action — le seul vrai renouvellement automatique hors stores.
App Store, Play Store	Le store renouvelle lui-même, selon ses règles.

3.6 Obtenir la coche



Le parcours est celui du volet 5 : photographier sa pièce d'identité, prendre un selfie en la tenant, envoyer. L'écran annonce *avant* l'envoi que les pièces sont chiffrées, consultées seulement par l'administration, et supprimées 90 jours après la décision. Réponse annoncée « sous 5 jours ouvrés ».

CHAPITRE 4

Du gratuit au payant

4.1 Trois phases

L'administration décide quand l'offre devient payante. Jusque-là, tout est gratuit ; ensuite, chacun dispose d'une période de grâce avant que les fonctionnalités ne soient réservées aux abonnés.



	Fonctionnalités	Coche
Gratuit	Pour tous. L'offre n'est pas proposée à l'achat.	Sur vérification d'identité seule.
Grâce	Pour tous. On peut s'abonner ; la première période commence à la fin de la grâce, les jours gratuits ne sont pas perdus.	Conservée par ceux qui l'ont. Elle tombe à la fin de la grâce s'ils ne s'abonnent pas.
Payant	Pour les abonnés, et pendant l'éventuel essai gratuit des nouveaux inscrits.	Identité vérifiée et abonnement actif.

Un seul bouton, pas trois. L'administrateur ne choisit pas une phase : il n'a qu'une action, « Activer le payant ». Elle déclenche la grâce, qui débouche d'elle-même sur la phase payante à sa date de fin. Il n'existe pas de bouton qui rende l'offre payante sur-le-champ : la grâce dure au moins 7 jours, pour que personne ne perde un accès sans avoir été prévenu. Le bandeau des trois phases, en tête de la page d'administration, est un repère, pas un sélecteur.

4.2 Ce que voit l'utilisateur au passage

- **À l'activation** : un message du compte officiel à tous les comptes, qui annonce la date et le prix. La carte du profil passe en mode « grâce ».
- **Sept jours avant la fin de la grâce** : un second message, aux seuls non-abonnés.
- **À la fin de la grâce** : les fonctionnalités se ferment pour les non-abonnés, avec le même délai de N jours avant suppression qu'une échéance ordinaire.

4.3 Les réglages de l'interrupteur

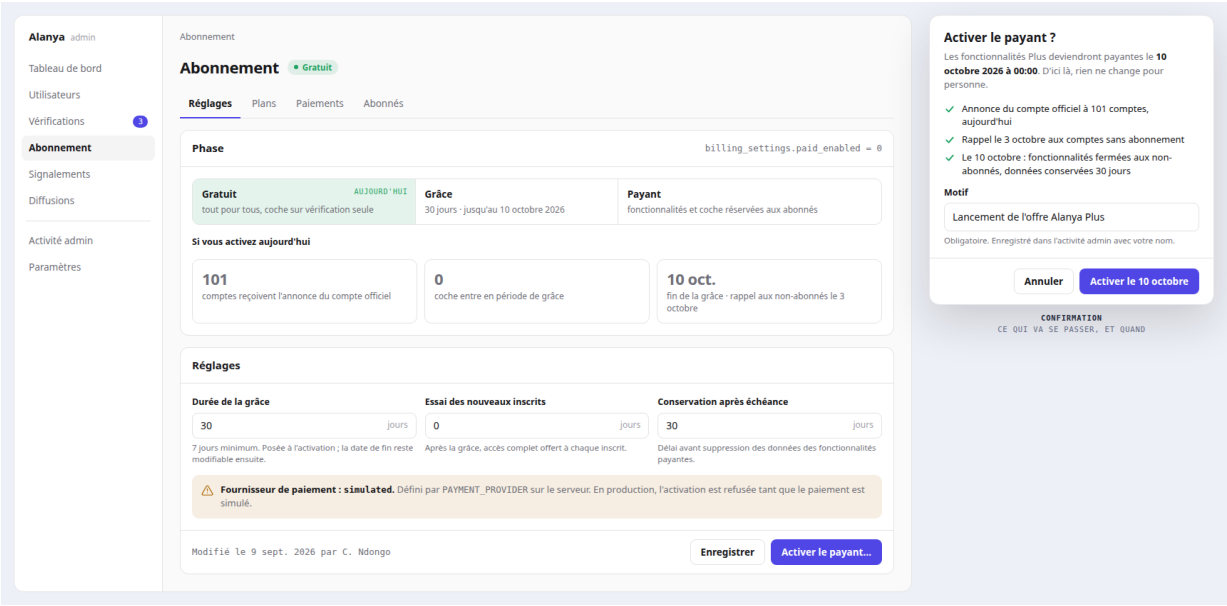
Réglage	Défaut	Effet
Payant activé	non	L'interrupteur lui-même.
Durée de la grâce	30 jours	7 jours au minimum. Posée à l'activation ; la date de fin reste modifiable ensuite, pour prolonger.
Essai gratuit des nouveaux inscrits	0 jour	Après la grâce, un inscrit reçoit cette durée d'accès complet.
Conservation échéance	après 30 jours	Le « N » avant suppression des données payantes.

Pas d'activation tant que le paiement est simulé. En production, l'interrupteur refuse de s'activer tant que le fournisseur de paiement est le simulateur. Sinon, n'importe qui obtiendrait un abonnement réel avec un paiement fictif.

CHAPITRE 5

Le parcours de l'administrateur

5.1 L'interrupteur et ses réglages



La page dit d'abord dans quelle phase on est, et ce que l'activation déclencherait : combien de comptes recevront l'annonce, combien de coches entreront en grâce. Activer demande une confirmation qui rappelle la date de fin de grâce.

Maquette interactive, administration. claude.ai/code/artifact/ffa8c829
Source versionnée : docs/architecture/maquettes/abonnement-admin.html.

5.2 Les plans

Plans

Réglages **Plans** Paiements Abonnés

Plan	Prix	Durée	Relance	État
Annuel plus_annuel · mis en avant	2 000 XAF	12 mois	30 j avant	Proposé
Mensuel plus_mensuel	250 XAF	1 mois	7 j avant	Proposé

Catalogue des fonctionnalités

Fonctionnalité	Payante	Livrée
Traduction des messages translation	☒	Oui
Sauvegarde des données backup	☒	Oui
Trajets de confiance trusted_trips	☒	Oui
Sonneries par liste list_ringtones	☒	Oui
Coche indigo verified_badge	☒	Oui
Personnalisation du style style	☒	À venir

Annuel mis en avant plus_annuel

Nom - français: Annuel, Nom - anglais: Yearly, Nom - chinois: 年度

Durée: 12 mois, Prix: 2 000 XAF, Relance: 30 jours avant

Fonctionnalités incluses

- ☒ Traduction des messages
- ☒ Trajets de confiance
- ☒ Coche indigo
- ☒ Sauvegarde des données
- ☒ Sonneries par liste
- ☐ Personnalisation du style (à venir)

☒ Proposé à l'achat ☒ Mis en avant dans l'offre

Produit App Store: à renseigner avec les stores

Produit Play Store: à renseigner avec les stores

APERÇU DANS L'APPLICATION

Annuel: 2 000 F / an, soit 167 F par mois, 4 MOIS OFFERTS

ⓘ Un nouveau prix vaut pour les prochains paiements. Les périodes déjà payées ne changent pas ; les abonnés en renouvellement automatique sont prévenus dans leur relance.

Retirer de l'offre Enregistrer

Pour chaque plan : son nom dans les trois langues, sa durée, son prix, son délai de relance, les fonctionnalités incluses (cases à cocher), s'il est mis en avant, s'il est proposé. Un aperçu montre la carte telle qu'elle apparaîtra dans l'application.

5.3 Un utilisateur

Utilisateurs · Marie Kouassi

M Marie Kouassi #2417 · 0042 8817 · @marie_k · inscrite le 11 mars 2026

Abonnement et coche Abonnée

Période en cours: Annuel · 10 oct. 2026 -- 10 oct. 2027 payée pendant la grâce, commence à sa fin

Renouvellement: Automatique Orange Money · +237 6+ -- 34 00

Identité: Vérifiée le 24 sept. 2026 par C. Ndongo · dossier #418 · nom vérifié « Marie Kouassi »

Coche: Active jusqu'au 10 oct. 2027

Offrir un abonnement... Révoquer la coche...

Socle de compte

Genre de compte: Personnel

L'état de vérification n'est plus modifiable ici : il découle du dossier d'identité, de l'abonnement et des révocations.

Paiements

Date	Montant	Moyen	Référence	Statut
22/09/2026 19:31	2 000 F	Orange Money	SIM-8F21C9	Réussi
22/09/2026 19:28	2 000 F	Orange Money	SIM-8F20A1	Refusé

Périodes

Du	Au	Plan	Origine
10 oct. 2026	10 oct. 2027	Annuel	Paiement SIM-8F21C9

Offrir un abonnement

Une période s'ajoute après celle en cours. Aucun paiement n'est créé.

Durée: 1 mois, 3 mois, 12 mois

Nouvelle échéance: 10 janvier 2028

Motif: Testeuse de la version bêta — compensation

Obligatoire, visible dans l'historique et l'activité admin.

Annuler Offrir 3 mois

GESTE COMMERCIAL
UNE PÉRIODE DE PLUS, JAMAIS UN FAUX PAIEMENT

La fiche utilisateur gagne une carte « Abonnement et coche » : la période en cours, l'historique des périodes et des paiements, l'état du dossier d'identité. Deux actions : **offrir** un abonnement (durée et motif obligatoires — un geste commercial, un partenaire, un testeur) et **révoquer** la coche (motif obligatoire). Le sélecteur « État de vérification » de la carte « Socle » disparaît : l'état se déduit des décisions, il ne se saisit plus.

5.4 Les dossiers d'identité

L'écran d'instruction est celui du volet 5 : ce que le compte déclare à gauche, les pièces à droite, trois issues — approuver, refuser avec motif, demander une pièce. Une file dédiée les rassemble, avec un compteur dans le menu.

CHAPITRE 6

Le paiement : simulé aujourd'hui, réel demain

6.1 Un faux agrégateur, un vrai parcours

Tant qu'aucun agrégateur n'est choisi, le serveur en *imite* un : il reçoit la demande, fait patienter quelques secondes, puis répond — le plus souvent par un succès. Certains numéros de test provoquent volontairement un échec, un refus ou une absence de réponse, pour que chaque cas soit vu à l'écran avant d'arriver en vrai.

Le jour où l'agrégateur est choisi, on écrit le module qui lui parle et on le met à la place du simulateur. L'application, l'administration, les abonnements et la coche ne changent pas.

6.2 Les stores

Apple et Google imposent leurs règles. L'application sera publiée sur l'App Store et le Play Store. Pour débloquent des fonctionnalités numériques, Apple impose son propre système de paiement, et Google le sien dans la plupart des pays — avec une commission de 15 à 30 %. Mobile money et, plus tard, porte-monnaie ne seront donc pas forcément proposés partout dans l'application. La conception l'absorbe : un store est un fournisseur de paiement de plus, au même endroit que le simulateur. Mais la règle exacte pour le Cameroun, et le prix affiché sur chaque store, restent à **décider avant de brancher un paiement réel.**

PARTIE II

Approche technique

CHAPITRE 7

Ce qui existe

Rien de commercial dans les trois dépôts. Aucune table, route, dépendance ni chaîne de traduction ne parle de plan, d'abonnement, de paiement ou de droit d'accès — ni dans le code, ni dans l'historique git, toutes branches comprises. La base de production ne contient aucune table de ce genre. Les tables `subscription` et `payment_transaction` du volet 5 n'ont jamais été créées.

Aucun réglage global partagé. L'application ne lit aucune configuration distante au démarrage. Côté serveur, les réglages de l'administration (`src/controllers/admin/settings.js`) sont écrits dans un *fichier* local, `data/app-settings.json`; `src/constants/tripPolicy.js` le dit en commentaire : « il n'existe pas de table de réglages globale ». Un fichier local diverge dès qu'il y a deux instances — et le passage à plusieurs instances est préparé (adaptateur Redis). L'interrupteur du payant doit donc vivre en base.

7.1 Les fonctionnalités à verrouiller

Fonctionnalité	Où elle vit	État dans le code
Traduction	téléphone	ML Kit sur l'appareil (<code>message_translation_service.dart</code>). Le serveur n'intervient pas.
Sauvegarde	téléphone + clé serveur	Serveur en place (<code>/api/backup</code> , migrations 078–079). Le code mobile n'existe que sur <code>origin/partition</code> (commit <code>c1ebc73</code>), pas encore fusionné. Les archives vont sur l'appareil ou Google Drive, jamais chez Alanya.
Trajets de confiance	serveur + téléphone	Complet (<code>/api/trips</code> , socket, jobs, migration 051).
Sonneries par liste	téléphone, <i>y compris natif</i>	Assignation synchronisée (migration 055) ; résolution en Dart <i>et</i> en Kotlin quand l'application est fermée (<code>CallIncomingHelper.kt</code> , <code>MessageNotificationHelper.kt</code>).
Style	téléphone	Seul le thème clair/sombre/système existe. Rien à verrouiller aujourd'hui.

CHAPITRE 8

Modèle de données

8.1 Les principes

Principe	Conséquence
Une ligne par période payée	subscription_period n'est jamais modifiée, seulement ajoutée. Être abonné = une période couvre l'instant présent. Renouveler en avance, changer de durée, offrir un mois : ce sont toutes des lignes de plus, sans machine à états à tenir.
Le catalogue est en base	Plans, prix, contenus et libellés se modifient depuis l'administration. Les <i>codes</i> de fonctionnalité, eux, sont connus du code : c'est lui qui pose les verrous.
Un seul réglage global	billing_settings n'a qu'une ligne, garantie par une contrainte CHECK.
Le paiement ne crée rien tant qu'il n'est pas confirmé	Une période naît de la <i>confirmation</i> d'un paiement, jamais de sa demande.

8.2 Migration 080_abonnement.sql

```
1 -- 080_abonnement.sql -- offre, periodes, paiements, reglage global
2 -- Requiert 038 (socle) et 039 (job_queue). Application manuelle.
3
4 CREATE TABLE IF NOT EXISTS feature (
5   code          VARCHAR(40) NOT NULL COMMENT 'Code connu du code
6     applicatif',
7   name_i18n     JSON          NOT NULL COMMENT
8     '{"fr":...,"en":...,"zh":...}',
9   description_i18n JSON        NULL,
10  is_paid        TINYINT       NOT NULL DEFAULT 1
11                  COMMENT '0 = gratuite pour tous, meme en phase
12                  payante',
13  is_available   TINYINT       NOT NULL DEFAULT 1
14                  COMMENT '0 = annoncee, pas encore livree',
15  sort_order     SMALLINT      NOT NULL DEFAULT 0,
16  updated_at     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
17                  ON UPDATE CURRENT_TIMESTAMP,
18  PRIMARY KEY (code)
19 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
20
21 CREATE TABLE IF NOT EXISTS plan (
22   id             INT           NOT NULL AUTO_INCREMENT,
23   code           VARCHAR(40)   NOT NULL,
24   name_i18n      JSON          NOT NULL,
```

```

22     duration_months      TINYINT      NOT NULL COMMENT '1 = mensuel,
        12 = annuel',
23     price_amount        INT           NOT NULL
24                        COMMENT 'Unites entieres : le XAF n a pas
        de sous-unite',
25     currency            CHAR(3)       NOT NULL DEFAULT 'XAF',
26     reminder_days       SMALLINT      NOT NULL COMMENT 'Relance avant
        echeance',
27     is_active            TINYINT       NOT NULL DEFAULT 1,
28     is_featured          TINYINT       NOT NULL DEFAULT 0,
29     sort_order           SMALLINT      NOT NULL DEFAULT 0,
30     store_product_ios    VARCHAR(100) NULL,
31     store_product_android VARCHAR(100) NULL,
32     updated_by           INT           NULL,
33     created_at           DATETIME      NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
34     updated_at           DATETIME      NOT NULL DEFAULT
        CURRENT_TIMESTAMP
35                                ON UPDATE CURRENT_TIMESTAMP,
36     PRIMARY KEY (id),
37     UNIQUE KEY uq_plan_code (code),
38     CONSTRAINT fk_plan_updater FOREIGN KEY (updated_by)
39     REFERENCES users(alanyaID) ON DELETE SET NULL
40 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
41
42 CREATE TABLE IF NOT EXISTS plan_feature (
43     plan_id      INT      NOT NULL,
44     feature_code VARCHAR(40) NOT NULL,
45     PRIMARY KEY (plan_id, feature_code),
46     CONSTRAINT fk_pf_plan FOREIGN KEY (plan_id) REFERENCES plan(id)
47     ON DELETE CASCADE,
48     CONSTRAINT fk_pf_feature FOREIGN KEY (feature_code) REFERENCES
        feature(code)
49 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
50
51 CREATE TABLE IF NOT EXISTS billing_settings (
52     id                TINYINT NOT NULL DEFAULT 1,
53     paid_enabled      TINYINT NOT NULL DEFAULT 0,
54     activated_at      DATETIME NULL,
55     grace_until       DATETIME NULL,
56     deactivated_at    DATETIME NULL,
57     default_grace_days SMALLINT NOT NULL DEFAULT 30,
58     trial_days        SMALLINT NOT NULL DEFAULT 0,
59     retention_days    SMALLINT NOT NULL DEFAULT 30
60                        COMMENT 'Conservation des donnees payantes
        apres echeance',
61     updated_by        INT      NULL,
62     updated_at        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
63                        ON UPDATE CURRENT_TIMESTAMP,
64     PRIMARY KEY (id),
65     CONSTRAINT ck_billing_singleton CHECK (id = 1),
66     -- Pas de passage au payant sans preavis : 7 jours de grace au
        minimum.
67     CONSTRAINT ck_billing_grace CHECK (default_grace_days >= 7),
68     CONSTRAINT fk_billing_updater FOREIGN KEY (updated_by)
69     REFERENCES users(alanyaID) ON DELETE SET NULL
70 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
71
72 INSERT IGNORE INTO billing_settings (id) VALUES (1);

```

Listing 8.1 – Catalogue et réglage global

```

1 CREATE TABLE IF NOT EXISTS payment (
2   id BIGINT NOT NULL AUTO_INCREMENT,
3   alanyaID INT NOT NULL,
4   plan_id INT NOT NULL,
5   provider VARCHAR(32) NOT NULL
6     COMMENT 'simulated, <agregateur>, apple, google,
7     wallet',
8   channel VARCHAR(32) NOT NULL
9     COMMENT 'orange_money, mtn_momo, store, wallet',
10  msisdn VARCHAR(20) NULL,
11  amount INT NOT NULL COMMENT 'Prix du plan AU
12  MOMENT du paiement',
13  currency CHAR(3) NOT NULL,
14  purpose TINYINT NOT NULL DEFAULT 0
15     COMMENT '0=souscription 1=renouvellement
16     2=renouvellement auto',
17  status TINYINT NOT NULL DEFAULT 0
18     COMMENT '0=cree 1=en attente 2=reussi 3=echoue
19     4=expire 5=rembourse',
20  idempotency_key VARCHAR(80) NOT NULL,
21  provider_ref VARCHAR(120) NULL,
22  failure_code VARCHAR(40) NULL,
23  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
24  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
25    ON UPDATE CURRENT_TIMESTAMP,
26  confirmed_at DATETIME NULL,
27  PRIMARY KEY (id),
28  UNIQUE KEY uq_payment_idem (idempotency_key),
29  -- Plusieurs NULL sont admis : un paiement n'a pas de reference
30  -- tant que le fournisseur ne l'a pas acceptee.
31  UNIQUE KEY uq_payment_ref (provider, provider_ref),
32  KEY idx_payment_user (alanyaID, created_at),
33  KEY idx_payment_pending (status, created_at),
34  CONSTRAINT fk_payment_user FOREIGN KEY (alanyaID)
35    REFERENCES users(alanyaID) ON DELETE CASCADE,
36  CONSTRAINT fk_payment_plan FOREIGN KEY (plan_id) REFERENCES plan(id)
37 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
38
39 -- Journal brut de tout ce que les fournisseurs envoient, signature
40 -- valide ou non. C'est lui qu'on relit quand un paiement est
41 -- conteste.
42
43 CREATE TABLE IF NOT EXISTS payment_event (
44   id BIGINT NOT NULL AUTO_INCREMENT,
45   payment_id BIGINT NULL COMMENT 'NULL si la reference est
46   inconnue',
47   provider VARCHAR(32) NOT NULL,
48   event_type VARCHAR(40) NOT NULL,
49   signature_ok TINYINT NOT NULL,
50   payload JSON NOT NULL,
51   received_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
52   PRIMARY KEY (id),
53   KEY idx_pe_payment (payment_id, received_at),
54   CONSTRAINT fk_pe_payment FOREIGN KEY (payment_id)
55     REFERENCES payment(id) ON DELETE SET NULL

```

```

49 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
50
51 CREATE TABLE IF NOT EXISTS subscription_period (
52   id          BIGINT          NOT NULL AUTO_INCREMENT,
53   alanyaID    INT             NOT NULL,
54   plan_id     INT             NOT NULL,
55   starts_at   DATETIME        NOT NULL,
56   ends_at     DATETIME        NOT NULL,
57   source      TINYINT         NOT NULL COMMENT '0=paiement 1=essai
      2=offert 3=compensation',
58   payment_id  BIGINT          NULL,
59   granted_by  INT             NULL COMMENT 'Administrateur, pour une
      periode offerte',
60   reason      VARCHAR(255)    NULL COMMENT 'Obligatoire si source = 2',
61   created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
62   PRIMARY KEY (id),
63   -- Un paiement confirme deux fois ne cree pas deux periodes.
64   UNIQUE KEY  uq_period_payment (payment_id),
65   KEY idx_period_user (alanyaID, ends_at),
66   CONSTRAINT fk_period_user FOREIGN KEY (alanyaID)
67     REFERENCES users(alanyaID) ON DELETE CASCADE,
68   CONSTRAINT fk_period_plan FOREIGN KEY (plan_id) REFERENCES plan(id),
69   CONSTRAINT fk_period_payment FOREIGN KEY (payment_id)
70     REFERENCES payment(id) ON DELETE SET NULL,
71   CONSTRAINT fk_period_granter FOREIGN KEY (granted_by)
72     REFERENCES users(alanyaID) ON DELETE SET NULL
73 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
74
75 -- Une ligne par utilisateur ayant eu au moins une periode. Ce qui est
76 -- deduit des periodes y est denormalise pour les balayages.
77 CREATE TABLE IF NOT EXISTS subscriber (
78   alanyaID    INT             NOT NULL,
79   current_end  DATETIME        NULL COMMENT 'Fin de la derniere periode',
80   auto_renew   TINYINT         NOT NULL DEFAULT 0,
81   renew_plan_id INT           NULL COMMENT 'Duree choisie pour le
      prochain renouvellement',
82   renew_channel VARCHAR(32)    NULL,
83   renew_msisdn VARCHAR(20)    NULL,
84   purge_after  DATETIME        NULL COMMENT 'Pose a l echeance : +
      retention_days',
85   purged_at    DATETIME        NULL,
86   updated_at   DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP
87               ON UPDATE CURRENT_TIMESTAMP,
88   PRIMARY KEY (alanyaID),
89   KEY idx_subscriber_end (current_end),
90   KEY idx_subscriber_purge (purge_after, purged_at),
91   CONSTRAINT fk_subscriber_user FOREIGN KEY (alanyaID)
92     REFERENCES users(alanyaID) ON DELETE CASCADE,
93   CONSTRAINT fk_subscriber_plan FOREIGN KEY (renew_plan_id)
94     REFERENCES plan(id) ON DELETE SET NULL
95 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Listing 8.2 – Paiements, périodes, abonné

```

1 INSERT IGNORE INTO plan (code, name_i18n, duration_months,
2   price_amount,
   currency, reminder_days, is_featured,
   sort_order) VALUES

```

```

3  ('plus_mensuel', JSON_OBJECT('fr', 'Mensuel', 'en', 'Monthly'), 1,
   250, 'XAF', 7, 0, 10),
4  ('plus_annuel', JSON_OBJECT('fr', 'Annuel', 'en', 'Yearly'), 12,
   2000, 'XAF', 30, 1, 20);
5
6  -- Les deux plans incluent toutes les fonctionnalités payantes.
7  INSERT IGNORE INTO plan_feature (plan_id, feature_code)
8  SELECT p.id, f.code FROM plan p CROSS JOIN feature f WHERE
   f.is_paid = 1;
9
10 INSERT IGNORE INTO scheduler_leases (name) VALUES ('billing_sweep');

```

Listing 8.3 – Contenu initial

Le catalogue des fonctionnalités est semé dans le même fichier. Les libellés complets, avec leurs accents et leur traduction chinoise, y figurent ; ce tableau en donne les codes :

Code	Payante	Disponible	Libellé français
translation	oui	oui	Traduction des messages
backup	oui	oui	Sauvegarde des données
trusted_trips	oui	oui	Trajets de confiance
list_ringtones	oui	oui	Sonneries par liste
style	oui	non	Personnalisation du style
verified_badge	oui	oui	Coche indigo

La coche est une fonctionnalité du catalogue. Mettre `verified_badge` au catalogue rend son inclusion réglable comme le reste : un futur plan business pourrait l'inclure, un plan d'entrée de gamme non. La règle d'affichage reste celle du chapitre 10 : identité vérifiée *et* droit à la fonctionnalité.

Deux pièges MySQL déjà rencontrés sur ce schéma. `users.alanyaID` est un INT signé : toutes les clés étrangères ci-dessus le sont aussi, sans quoi l'erreur 3780 bloque la migration. Et le `XAF` n'a pas de sous-unité : 250 F s'écrit 250, pas 25000. Certains agrégateurs attendent des centimes d'office ; la conversion appartient à leur module, jamais au modèle.

8.3 Migration 081_verification.sql

Elle reprend le DDL du volet 5 (`verification_request`, `verification_document`, `verification_document_access`, prévu sous le numéro 027 jamais utilisé), avec trois écarts :

Écart	Pourquoi
Statut 5 = révoqué sur la demande	La révocation porte sur une décision déjà prise ; elle garde son auteur et son motif sur la même ligne.
Colonne <code>name_at_approval</code>	Le nom vérifié. S'il ne correspond plus au nom affiché, la coche tombe jusqu'à nouvel examen.
Plus de <code>subscription</code> ni de <code>payment_transaction</code>	Remplacées par la migration 080.

CHAPITRE 9

Droits d'accès

9.1 Une seule fonction décide

Tout ce qui se demande « cet utilisateur a-t-il droit à cette fonctionnalité ? » passe par `entitlementsFor`. Aucun écran, aucune route ne recalcule la règle de son côté.

```
1  async function entitlementsFor(alanyaID, now = new Date()) {
2    const s = await getBillingSettings();           // une ligne, cache 30 s
3    const phase = !s.paid_enabled ? 'free'
4                : now < s.grace_until ? 'grace' : 'paid';
5
6    const catalog = await getFeatures();           // catalogue, cache 30 s
7    const period = await currentPeriod(alanyaID, now); // null si
8    // aucune
9    const included = period ? await featuresOfPlan(period.plan_id) : [];
10   const exempt = await isStaffOrOfficial(alanyaID); // admins,
11   // compte officiel
12
13   const features = {};
14   for (const f of catalog) {
15     features[f.code] = f.is_available === 1 && (
16       f.is_paid === 0 || phase !== 'paid' || exempt ||
17       included.includes(f.code));
18   }
19   return {
20     phase,
21     graceUntil: s.grace_until,
22     period, // plan, debut, fin,
23     // source
24     features,
25     // Jusqu'a quand le telephone peut se fier a ce resultat sans
26     // reseau.
27     validUntil: earliest(period && period.ends_at,
28                           phase === 'grace' ? s.grace_until : null,
29                           addDays(now, 7)),
30   };
31 }
```

Listing 9.1 – `src/services/billing/entitlements.js`

L'essai gratuit n'a pas de règle à part : c'est une période de `source = 1`, créée à l'inscription quand `trial_days > 0` et que la phase est payante. Un abonnement offert est une période de `source = 2`.

9.2 Ce que reçoit le téléphone

```
{
  "phase": "grace",
  "graceUntil": "2026-10-12T00:00:00Z",
  "period": { "plan": "plus_annuel", "endsAt": "2027-10-12T00:00:00Z",
    "autoRenew": true },
  "features": { "translation": true, "backup": true, "trusted_trips":
    true,
    "list_ringtones": true, "style": false,
    "verified_badge": true },
  "validUntil": "2026-09-17T08:00:00Z"
}
```

Listing 9.2 – Charge utile entitlements

Elle arrive par trois chemins : dans GET /api/auth/me, par GET /api/billing/me, et poussée par l'événement socket entitlements:updated à chaque paiement, échéance ou changement de réglage. Le téléphone la garde en cache avec son validUntil : la traduction, qui fonctionne sans réseau, doit continuer à fonctionner sans réseau.

9.3 Deux verrous, selon l'endroit où vit la fonctionnalité

Côté serveur	Middleware requireFeature('trusted_trips') sur les routes concernées. Refus en 403 avec le code SUBSCRIPTION_REQUIRED et le code de la fonctionnalité.
Côté téléphone	EntitlementService.has(Feature.translation) lu dans le cache. Pour ce que le serveur ne voit pas — traduction, sonneries, style — c'est le seul verrou.

Le refus n'est pas une erreur. SUBSCRIPTION_REQUIRED rejoint docs/error-codes.md, mais l'entonnoir d'erreurs de l'application (lib/core/errors/error_presenter.dart) ne l'affiche pas comme un message : il ouvre le panneau de l'offre sur la fonctionnalité demandée. Un seul endroit traduit donc tous les refus en proposition, quel que soit l'écran qui les a reçus.

Un verrou côté téléphone se contourne. Une application modifiée peut ignorer le cache et traduire sans abonnement. Pour une fonctionnalité à 250 F qui tourne entièrement sur l'appareil, c'est un risque accepté : le fermer demanderait de faire passer la traduction par le serveur, ce qui coûterait plus que ce qu'on protège.

CHAPITRE 10

La coche

10.1 Une seule écriture

`users.verification_status` et `users.verified_until` ne sont plus jamais saisis. Une seule fonction les écrit, appelée à chaque événement qui peut les changer : décision sur un dossier, paiement confirmé, échéance, fin de grâce, changement de nom, changement de réglage.

```
1  async function recomputeVerification(alanyaID, conn) {
2    const req = await latestRequest(alanyaID, conn); // hors dossiers
      annules
3    const ent = await entitlementsFor(alanyaID);
4    let status = V.NON_DEMANDE;
5    let until = null;
6
7    if (req && req.status === R.REVOQUE)      status = V.REVOQUE;
8    else if (req && req.status === R.REFUSE)   status = V.REFUSE;
9    else if (req && req.status !== R.APPROUVE) status = V.EN_COURS;
10   else if (req && nameChanged(req, alanyaID)) status = V.EN_COURS;
      // nouvel examen
11   else if (req && ent.features.verified_badge) {
12     status = V.VERIFIE;
13     until = ent.phase === 'free' ? null
14           : latest(ent.period && ent.period.ends_at,
15                 ent.phase === 'grace' ? ent.graceUntil : null);
16   } else if (req)                          status = V.EXPIRE;
17
18   await conn.execute(
19     'UPDATE users SET verification_status = ?, verified_until = ?
20     WHERE alanyaID = ?',
21     [status, until, alanyaID]);
22 }
```

Listing 10.1 – `src/services/billing/verification.js`

Corriger les constantes avant tout. Le backend déclare aujourd’hui `EXPIRE = 4` et `REVOQUE = 5` (`src/constants/accountTypes.js`, lignes 12–13), l’inverse de l’administration et du volet 1. Aucune donnée n’est touchée — en production, tous les comptes sont à 0 — mais la correction doit précéder la première écriture de ces valeurs.

10.2 Ce que la coche doit encore corriger dans l'existant

Constat	Correction
Un compte personnel vérifié n'affiche aucun badge (<code>account_badge.dart</code> , et son miroir <code>components/account-badge.tsx</code>)	Variante <code>cocheVerifiee</code> , indigo, prévue au volet 1 et jamais écrite.
Le socle manque dans la recherche, les contacts, les messages temps réel et le détail d'une conversation (qui le lit en base puis l'oublie)	Ajouter les deux champs aux quatre charges utiles.
Le sélecteur d'état de la carte « Socle » écrit n'importe quelle valeur, sans validation	Retiré : la route <code>PUT /admin/users/:id/socle</code> ne touche plus que <code>account_type</code> .

CHAPITRE 11

Le paiement

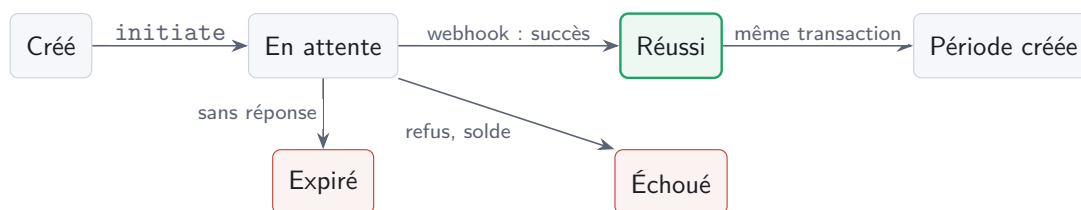
11.1 La couture

Tout fournisseur — le simulateur aujourd’hui, l’agrégateur demain, les stores, le porte-monnaie — est un module de trois fonctions. C’est le **seul** fichier à écrire pour en brancher un nouveau.

```
1 module.exports = {
2   name: 'simulated',
3   channels: ['orange_money', 'mtn_momo'],
4
5   // Demande le paiement au fournisseur. Ne confirme jamais rien :
6   // la confirmation n'arrive que par parseWebhook() ou fetchStatus().
7   async initiate({ payment, msisdn }) {
8     return { providerRef, status: 'pending', nextAction: { type:
9       'ussd_push' } };
10
11   // Verifie la signature et traduit l'evenement du fournisseur
12   // en { providerRef, outcome: 'succeeded' ou 'failed', failureCode
13     }.
14   async parseWebhook({ headers, rawBody }) { /* ... */ },
15
16   // Reconciliation, quand un webhook s'est perdu.
17   async fetchStatus(providerRef) { /* ... */ },
18 };
```

Listing 11.1 – src/services/payments/providers/<nom>.js

11.2 Le cycle d'un paiement



```
1 async function settlePayment(paymentId, outcome) {
2   const conn = await pool.getConnection();
3   try {
4     await conn.beginTransaction();
5     const [[p]] = await conn.execute(
6       'SELECT * FROM payment WHERE id = ? FOR UPDATE', [paymentId]);
```

```

7      if (p.status === P.REUSSI || p.status === P.ECHOUÉ) { // rejeu
          : rien
8      await conn.commit();
9      return p;
10     }
11     if (outcome.outcome !== 'succeeded') {
12         await markFailed(conn, p, outcome.failureCode);
13     } else {
14         const plan = await getPlan(conn, p.plan_id);
15         // max(maintenant, fin de la periode en cours, fin de la grace)
16         :
17         // payer en avance prolonge, payer pendant la grace ne la
18         // consomme pas.
19         const start = await nextStart(conn, p.alanyaID);
20         const end = addMonths(start, plan.duration_months);
21         await markSucceeded(conn, p);
22         await insertPeriod(conn, p, plan, start, end); //
23         // uq_period_payment
24         await upsertSubscriber(conn, p, end);
25         await recomputeVerification(p.alanyaID, conn);
26     }
27     await conn.commit();
28 } catch (err) {
29     await conn.rollback();
30     throw err;
31 } finally {
32     conn.release();
33 }
34 await afterSettle(paymentId); // push, socket, planification des
35 // relances
36 }

```

Listing 11.2 – settlePayment — le seul chemin vers une période

Trois protections contre le double crédit. Le verrou FOR UPDATE sérialise deux webhooks simultanés; le contrôle de statut rend le second inopérant; et `uq_period_payment` refuse physiquement une seconde période pour le même paiement, si les deux premières protections venaient à être contournées.

11.3 Le simulateur

`providers/simulated.js` répond comme un agrégateur : `initiate` renvoie « en attente », puis un job `payment_sim_callback` rappelle, quatre secondes plus tard, *la même fonction* que la route de webhook. Le parcours éprouvé aujourd’hui est donc celui qui servira demain, signature et idempotence comprises.

Numéro se termi- Réponse simulée nant par

00 (et tout autre)	Succès après 4 secondes.
01	Échec : solde insuffisant.
02	Refus de l'utilisateur sur son téléphone.
03	Aucune réponse : le paiement expire, la réconciliation le constate.
04	Succès envoyé deux fois — pour vérifier que la seconde est sans effet.

Le simulateur ne doit jamais encaisser en production. Le fournisseur actif est choisi par la variable d'environnement `PAYMENT_PROVIDER`, avec les secrets qui l'accompagnent. En production, l'activation du payant est refusée (409 `BILLING_PROVIDER_SIMULATED`) tant que ce fournisseur est `simulated`; le refus est testé.

11.4 Réconciliation

Un webhook peut se perdre. Toutes les cinq minutes, sous le bail `billing_sweep`, les paiements « en attente » depuis plus de deux minutes sont interrogés par `fetchStatus`; au-delà de trente minutes sans réponse, ils passent en « expiré ».

11.5 Les stores et le porte-monnaie

Fournisseur	Ce que fait son module
apple	StoreKit 2 dans l'application; le serveur vérifie la transaction signée auprès de l'App Store Server API et reçoit les <i>App Store Server Notifications</i> v2 sur la route de webhook.
google	Play Billing dans l'application; le serveur vérifie le jeton d'achat par l'API Google Play Developer et reçoit les notifications en temps réel via Pub/Sub.
wallet	Débit du solde dans la même transaction : <code>initiate</code> répond directement « réussi ».

Les produits des stores sont reliés aux plans par `store_product_ios` et `store_product_android`. Sur un store, le prix affiché vient du store, pas de `plan.price_amount` : c'est ce qui laisse ouverte la question du prix par canal.

CHAPITRE 12

Échéances et interrupteur

12.1 Des jobs posés à la bonne heure

La file de jobs (`src/services/jobQueue.js`) sait déjà exécuter un travail à une date précise (`runAfter`), dédoublonné par clé — les trajets s'en servent. Chaque période créée pose ses jobs ; chaque job relit l'état au moment de s'exécuter, si bien qu'un renouvellement survenu entre-temps les rend inopérants sans avoir à les annuler.

Job	Quand	Effet
<code>billing_reminder</code>	<code>fin - reminder_days</code>	Notification de relance.
<code>billing_autorenew</code>	<code>fin - 1 jour</code>	Si <code>auto_renew</code> : <code>initiate</code> sur le dernier canal utilisé.
<code>billing_expire</code>	<code>fin</code>	<code>recomputeVerification</code> , <code>purge_after = fin + retention_days</code> , notification.
<code>billing_purge_warning</code>	<code>purge_after - 7 jours</code>	Dernier avertissement.
<code>billing_purge</code>	<code>purge_after</code>	Suppression fonctionnalité par fonctionnalité.
<code>billing_grace_end</code>	<code>grace_until</code>	Recalcul des coches par lots de 500.

Un balayage quotidien, sous le bail `billing_sweep`, rattrape tout job perdu : il cherche les `subscriber` dont `current_end` est passée sans expiration enregistrée, et les `purge_after` échues sans `purged_at`. C'est le patron du `verificationScheduler`, qu'il remplace.

12.2 Les notifications

Cinq types rejoignent `NOTIFICATION_TYPES` (`src/notifications/notificationContract.js`) : `billing_reminder`, `billing_expired`, `billing_purge_warning`, `payment_succeeded`, `payment_failed`. Aucun ne passe outre « Ne pas déranger » : ce privilège reste réservé aux alertes de sûreté des trajets. Les annonces collectives — activation, fin de grâce — passent par une diffusion du compte officiel (`publishBroadcast`), avec un `clientId` qui la rend idempotente.

12.3 Les transitions de l'interrupteur

Action	Ce que fait le serveur
Activer	Refus si fournisseur simulé en production. Sinon <code>paid_enabled = 1</code> , <code>activated_at = maintenant</code> , <code>grace_until = maintenant + default_grace_days</code> ; diffusion d'annonce; job <code>billing_grace_end</code> ; diffusion de rappel à <code>grace_until - 7</code> jours, critère « sans période en cours ».
Prolonger la grâce	Nouvelle <code>grace_until</code> ; le job est replanifié par sa clé de dédoublement.
Désactiver	<code>paid_enabled = 0</code> , <code>deactivated_at = maintenant</code> . Tout redevient gratuit; les renouvellements automatiques sont suspendus.
Réactiver	Nouvelle grâce. Les abonnés dont une période couvrait <code>deactivated_at</code> reçoivent une période de compensation (<code>source = 3</code>) égale à la durée de la phase gratuite.

Chaque transition est une route dédiée (POST /admin/billing/activate...) plutôt qu'un PUT générique : le journal d'audit (`src/middleware/adminAudit.js`) enregistre ainsi un verbe explicite, avec le motif saisi.

CHAPITRE 13

Verrouillage, fonctionnalité par fonctionnalité

	Verrou	À l'échéance	Après N jours
Traduction	Téléphone : <code>_isEnabledFor</code> du service de traduction, le menu « Traduire », l'écran de réglages.	La traduction automatique s'arrête ; les bulles affichent l'original.	Suppression locale des modèles ML Kit et des traductions en cache.
Sauvegarde	Serveur : <code>GET /api/backup/key</code> (clé courante, pour une <i>nouvelle</i> sauvegarde) et <code>PUT /meta</code> . Téléphone : la planification s'arrête.	Plus de nouvelle sauvegarde. La restauration (<code>GET /key/:kid</code>) reste ouverte.	Voir l'alerte ci-dessous : rien à supprimer chez Alanya.
Trajets	Serveur : <code>POST /api/trips</code> et <code>POST /api/trips/sos</code> . Suivre, confirmer, prolonger, annuler et le SOS d'un trajet déjà lancé restent libres.	Un trajet en cours va à son terme, SOS compris. Plus de nouveau trajet ni de SOS isolé.	Suppression des trajets du compte (<code>trip</code> , <code>trip_point</code> , <code>trip_event</code>).
Sonneries par liste	Téléphone : <code>resolveCall</code> et <code>resolveMessage</code> renvoient <code>null</code> ; les helpers Kotlin lisent un drapeau miroir. Serveur : écriture des champs de son dans <code>PUT /api/contact-lists</code> .	Retour à la sonnerie globale ; le réglage est conservé.	Effacement des colonnes <code>*_sound*</code> de <code>contact_list</code> et des préférences locales.
Coche	Serveur : <code>recomputeVerification</code> .	Tombe.	Rien : l'identité reste vérifiée, les pièces suivent leur propre purge à 90 jours.

Les sonneries se décident aussi hors de Dart. Quand l'application est fermée, c'est le code natif Android qui choisit la sonnerie (`CallIncomingHelper.kt`, lignes 160–195; `MessageNotificationHelper.kt`, lignes 388–410), en relisant directement les préférences `flutter.list_ringtone_*`. Couper le verrou en Dart seulement laisserait les sonneries par liste actives pour tout appel reçu application fermée. Le service de droits écrit donc un drapeau `flutter.plus_list_ringtones` que les deux helpers consultent.

La sauvegarde n'appartient pas à Alanya. Les archives sont sur le téléphone ou le Google Drive de l'utilisateur ; le serveur ne détient que le secret qui dérive les clés. « Supprimer après N jours » n'a donc qu'un seul levier : refuser définitivement la clé de restauration — ce qui rendrait les archives de l'utilisateur illisibles pour toujours. La conception s'y refuse : faire une nouvelle sauvegarde est payant, relire la sienne ne l'est jamais. Écart validé : pour la sauvegarde, la purge à N jours ne s'applique pas.

Le SOS est dans l'offre, pas le SOS d'un trajet en cours. `POST /api/trips/sos` (`src/routes/trips.js`, ligne 156) déclenche une alerte sans trajet : il reçoit `requireFeature('trusted_trips')`. `POST /api/trips/:id/sos` (ligne 280) n'est *pas* verrouillé : un trajet lancé pendant l'abonnement garde son SOS jusqu'à sa fin, même si l'échéance tombe entre-temps. Les proches qui *suivent* un trajet n'ont besoin d'aucun abonnement.

CHAPITRE 14

API

14.1 Application

	Route	Objet
GET	/api/billing/offer	Phase, plans et fonctionnalités, dans la langue de l'utilisateur, canaux disponibles sur sa plateforme
GET	/api/billing/me	La charge utile <code>entitlements</code>
POST	/api/billing/checkout	{ <code>planCode</code> , <code>channel</code> , <code>msisdn</code> } — renvoie le paiement et l'action attendue
GET	/api/billing/payments/:id	Statut, en complément de l'événement socket <code>payment:updated</code>
GET	/api/billing/history	Périodes et paiements
PUT	/api/billing/preferences	{ <code>autoRenew</code> , <code>renewPlanCode</code> }
POST	/api/billing/store/:store	Jeton d'achat Apple ou Google, à vérifier (plus tard)

Les routes du dossier d'identité sont celles du volet 5 (`/api/verification`).

14.2 Fournisseurs

POST	/api/payments/webhook/:provider	Sans authentification de session : la signature du fournisseur fait foi. Tout est journalisé dans <code>payment_event</code> , y compris les appels à signature invalide.
-------------	---------------------------------	---

14.3 Administration

	Route	Permission
GET	/admin/billing/settings	billing.read
POST	/admin/billing/activate, /deactivate, /extend-grace	billing.settings
PUT	/admin/billing/settings	billing.settings (essai, conservation)
GET	/admin/billing/plans, /features	billing.read
POST	/admin/billing/plans	billing.plans
PUT	/admin/billing/plans/:id, /features/:code	billing.plans
GET	/admin/billing/payments, /subscribers	billing.read
GET	/admin/users/:id/billing	billing.read
POST	/admin/users/:id/billing/gift	billing.gift

billing.read rejoint les permissions de tout administrateur ; billing.settings, billing.plans et billing.gift sont réservées au super-administrateur (src/constants/adminRoles.js). Les décisions sur les dossiers d'identité suivent le volet 5 : tout administrateur, auteur tracé.

CHAPITRE 15

Impact fichier par fichier

15.1 Backend

Fichier	Nature	Objet
migrations/080_abonnement.sql	nouveau	Catalogue, périodes, paiements, réglage
migrations/081_verification.sql	nouveau	Dossiers et pièces (volet 5 révisé)
src/constants/accountTypes.js	modifié	Statuts 4 et 5 dans le bon ordre
src/constants/billing.js	nouveau	Statuts de paiement, sources de période
src/services/billing/settings.js	nouveau	Lecture en cache, transitions
src/services/billing/catalog.js	nouveau	Plans et fonctionnalités
src/services/billing/entitlements.js	nouveau	entitlementsFor
src/services/billing/verification.js	nouveau	recomputeVerification
src/services/billing/billingJobs.js	nouveau	Relance, échéance, purge, grâce
src/services/billing/featurePurge.js	nouveau	Purge par fonctionnalité
src/services/payments/paymentService.js	nouveau	checkout, settlePayment, réconciliation
src/services/payments/providers/simulated.js	nouveau	Le faux agrégateur
src/middleware/requireFeature.js	nouveau	Verrou serveur
src/controllers/billingController.js	nouveau	Routes de l'application
src/controllers/paymentWebhookController.js	nouveau	Webhooks
src/controllers/admin/billing.js	nouveau	Routes d'administration
src/routes/billing.js, src/routes/admin.js	nouveau, modifié	Montage
src/routes/trips.js, backup.js, contactLists.js	modifié	requireFeature
src/controllers/admin/users.js	modifié	/socle ne pose plus la vérification
src/controllers/authCustomController.js	modifié	entitlements dans /me
src/constants/adminRoles.js, src/middleware/adminAudit.js	modifié	Permissions et verbes d'audit
src/notifications/notificationContract.js	modifié	Cinq types
src/services/verificationScheduler.js	supprimé	Remplacé par billingJobs.js

Fichier	Nature	Objet
<code>server.js</code>	modifié	Routes, jobs, bail
<code>docs/error-codes.md</code>	modifié	SUBSCRIPTION_REQUIRED, BILLING_*, PAYMENT_*

15.2 Administration

Fichier	Objet
<code>app/(dashboard)/billing/page.tsx</code>	Phase, interrupteur, réglages
<code>app/(dashboard)/billing/plans/page.tsx</code> , <code>plans/[id]/page.tsx</code>	Plans et éditeur
<code>app/(dashboard)/billing/payments/page.tsx</code>	Paielements
<code>app/(dashboard)/billing/subscribers/page.tsx</code>	Abonnés et échéances
<code>app/(dashboard)/verifications/page.tsx</code> , <code>[id]/page.tsx</code>	File et instruction (volet 5)
<code>app/(dashboard)/users/[id]/page.tsx</code>	Carte « Abonnement et coche » ; sélecteur d'état retiré
<code>app/(dashboard)/layout.tsx</code>	Entrées de menu, compteur de dossiers
<code>components/billing/*</code> , <code>lib/api/billing.ts</code> , <code>hooks/useBilling.ts</code> , <code>types/index.ts</code>	Composants, appels, types
<code>components/account-badge.tsx</code>	Coche personnelle

15.3 Application

Fichier	Objet
<code>lib/api/billing_api.dart</code>	Appels
<code>lib/core/services/billing/entitlement_service.dart</code>	Cache des droits, <code>validUntil</code> , drapeaux natifs
<code>lib/screens/billing/plus_offer_screen.dart</code>	Offre
<code>lib/screens/billing/checkout_screen.dart</code>	Paielement et attente
<code>lib/screens/billing/subscription_screen.dart</code>	Mon abonnement
<code>lib/screens/profile/verification_screen.dart</code>	Obtenir la coche (volet 5)
<code>lib/widgets/billing/plus_card.dart</code>	Carte du profil, cinq états
<code>lib/widgets/billing/paywall_sheet.dart</code>	Panneau de fonctionnalité verrouillée
<code>lib/screens/profile/profile_screen.dart</code>	Carte sous <code>_ProfileHeader</code> , entrée de menu
<code>lib/core/errors/error_presenter.dart</code>	SUBSCRIPTION_REQUIRED ouvre le panneau

Fichier	Objet
lib/core/services/translation/ message_translation_service.dart	Verrou traduction
lib/core/services/list_ringtone_preferences.dart	Verrou sonneries
android/.../CallIncomingHelper.kt, MessageNotificationHelper.kt	Drapeau miroir
lib/screens/trips/	Panneau au lieu de l'écran de départ
lib/widgets/common/account_badge.dart	Variante cocheVerifiee
lib/l10n/app_fr.arb, app_en.arb, app_zh.arb	Chaînes

CHAPITRE 16

Ordre de livraison

Lot	Contenu	Ce qu'il rend possible
A	Corrections du socle	Statuts dans le bon ordre, coche personnelle visible, socle présent dans toutes les charges utiles.
B	Catalogue et interrupteur	Migration 080, droits d'accès, écrans d'administration des plans et des réglages. Tout reste gratuit : rien ne change pour l'utilisateur.
C	Offre et paiement simulé	Carte du profil, offre, paiement, panneau, verrous des quatre fonctionnalités. Testable de bout en bout.
D	Échéances	Relances, renouvellement automatique, expiration, purge.
E	Coche	Migration 081, coffre à documents, dossier, instruction.
F	Paieement réel	Agrégateur, stores, puis porte-monnaie — après les décisions sur les stores.

B peut partir en production avant C : avec l'interrupteur éteint, le catalogue est inerte. E ne dépend que de A ; il peut avancer en parallèle de C et D.

CHAPITRE 17

Points de vigilance et questions ouvertes

17.1 Décisions prises

Sujet	Décision
Nom de l'offre	« Alanya Plus », définitif.
Sauvegarde	Toujours restaurable ; la purge à N jours ne s'y applique pas.
SOS	Dans l'offre. Le SOS d'un trajet déjà lancé reste disponible jusqu'à sa fin.
Retour au gratuit	Les abonnés sont compensés de la durée de la phase gratuite (période <code>source</code> = 3).
Vérification d'identité	Valable sans limite de durée. Seul un changement de nom provoque un nouvel examen.
Remboursements	Aucun. Les conditions générales doivent le dire avant le premier paiement.
Durée de grâce	7 jours au minimum (contrainte <code>ck_billing_grace</code>).

« **Aucun remboursement** » ne vaut que pour **Alanya**. Apple et Google remboursent parfois un achat de leur propre chef, sans demander l'avis de l'éditeur. Leur notification de remboursement arrive par la route de webhook : le paiement passe en statut 5, et la période qu'il avait créée est close à la date du remboursement. Sans ce traitement, un utilisateur remboursé par son store garderait son abonnement.

17.2 Questions ouvertes

1. **Prix sur les stores** : même prix, prix majoré, ou autre ? Reporté par décision ; le modèle le permet sans changement.
2. **Règles des stores pour le Cameroun** : le Play Store autorise-t-il un paiement tiers pour des fonctionnalités numériques ? L'application iOS peut-elle seulement mentionner le mobile money ? À vérifier *avant* le lot F.
3. **Fiscalité** : le prix affiché est-il toutes taxes comprises ? La TVA camerounaise sur les services numériques est à établir avec un comptable *avant* l'activation du payant. Elle ne change pas le modèle : un prix toutes taxes comprises reste un entier dans `plan.price_amount` ; seule la facture, si elle est exigée, devra détailler la part de taxe.
4. **Comptes business** : auront-ils leurs propres plans ? Le catalogue le permet (un plan peut inclure d'autres fonctionnalités), rien n'est prévu à ce stade.

17.3 Ce qui ne doit pas être oublié

Le verrou natif des sonneries. Sans le drapeau miroir, les sonneries par liste restent actives application fermée — le cas le plus fréquent pour un appel entrant.

Le changement de nom. Il doit appeler `recomputeVerification`. C'est ce qui empêche d'acheter une coche sous son nom puis de se renommer.

Le simulateur en production. L'activation refuse le fournisseur simulé en production ; ce refus doit avoir son test, dans les scripts lancés par la CI.